

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1704

COURSE OUTCOME COVERED

CO2: *Perform dry run testing to trace the execution of algorithms, analyze intermediate steps, and verify the correctness of processes. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2

Total Marks:20

Annexure A

DRY RUN TEST

Code of Conduct:

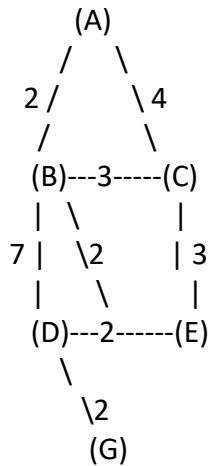
I Bhavya S(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Bhavya S

Question 1: Dry Run of A Search Algorithm (10 Marks)*

Consider the following graph.



Heuristic Values

Node h(n)

A	7
B	6
C	4
D	3
E	2
G	0

Task:

Perform a **dry run of the A* Search algorithm** from **A** to **G**.

For each iteration, show:

- Current node
- Open List
- Closed List
- $g(n)$
- $h(n)$
- $f(n) = g(n) + h(n)$

Finally,

1. Identify the optimal path.
2. Calculate the total path cost.

Aim

To find the shortest path from the start node A to the goal node G using the A* Search Algorithm.

Theory

The A* (A-Star) Search Algorithm is an informed search algorithm that finds the shortest path between a start node and a goal node. It combines the actual cost from the start node with a heuristic estimate of the remaining cost to reach the goal.

The evaluation function is:

$$\begin{aligned} &[\\ f(n) &= g(n) + h(n) \\ &] \end{aligned}$$

Where:

- **$g(n)$** = Actual cost from the start node to node n .
- **$h(n)$** = Heuristic value (estimated cost from node n to the goal).
- **$f(n)$** = Total estimated cost.

The algorithm always selects the node with the smallest $f(n)$ from the Open List.

Given Graph

Edges:

- $A \rightarrow B = 2$
- $A \rightarrow C = 4$
- $B \rightarrow C = 3$
- $B \rightarrow D = 7$
- $B \rightarrow E = 2$
- $C \rightarrow E = 3$
- $D \rightarrow E = 2$
- $E \rightarrow G = 2$

Heuristic Values

Node $h(n)$

A 7

Node h(n)

B 6

C 4

D 3

E 2

G 0

Dry Run

Initially,

- Open List = {A}
- Closed List = { }
- $g(A)=0$
- $h(A)=7$
- $f(A)=7$

Iteration 1**Current Node**

A

Since A is the only node in the Open List, it is selected.

Move A to the Closed List.

Expand node A.

Neighbours:

Node B

$$g(B)=0+2=2$$

$$h(B)=6$$

$$f(B)=2+6=8$$

Node C

$$g(C)=0+4=4$$

$$h(C)=4$$

$$f(C)=4+4=8$$

Lists after Expansion

Open List

Node g h f

B 2 6 8

C 4 4 8

Closed List

A

Iteration 2

Choose the node with the smallest f value.

Both B and C have $f=8$.

Select B (leftmost node).

Move B to Closed List.

Expand B.

Neighbours:

C

$$\text{New } g = 2+3=5$$

$$\text{Existing } g(C)=4$$

Since 4 is smaller,

Ignore the new path.

D

$$g(D)=2+7=9$$

$$h(D)=3$$

$$f(D)=12$$

E

$$g(E)=2+2=4$$

$$h(E)=2$$

$$f(E)=6$$

Lists after Expansion

Open List

Node g h f

E 4 2 6

C 4 4 8

D 9 3 12

Closed List

A, B

Iteration 3

Node with minimum f value is E.

Move E to Closed List.

Expand E.

Neighbour:

G

$$g(G)=4+2=6$$

$$h(G)=0$$

$$f(G)=6$$

Neighbour C

$$\text{New } g = 4+3=7$$

$$\text{Existing } g(C)=4$$

Ignore.

Lists after Expansion

Open List

Node g h f

G 6 0 6

C 4 4 8

D 9 3 12

Closed List

A, B, E

Iteration 4

Node with minimum f value is G.

Since G is the goal node, stop the search.

Closed List

A, B, E, G

Iteration Summary

Iteration	Current Node	g(n)	h(n)	f(n)	Open List	Closed List
1	A	0	7	7	B(8), C(8)	A
2	B	2	6	8	E(6), C(8), D(12)	A, B
3	E	4	2	6	G(6), C(8), D(12)	A, B, E
4	G	6	0	6	C(8), D(12)	A, B, E, G

Optimal Path

Using the parent nodes,

A → B → E → G

Total Path Cost

A → B = 2

$B \rightarrow E = 2$

$E \rightarrow G = 2$

Total Cost = 2 + 2 + 2 = 6

Python Code:

```
import heapq

graph = {}

heuristic = {}

# Number of nodes
n = int(input("Enter number of nodes: "))
print("\nEnter node names:")
nodes = []

for i in range(n):
    node = input()
    nodes.append(node)
    graph[node] = []

# Number of edges
e = int(input("\nEnter number of edges: "))
print("\nEnter edges (Source Destination Cost):")

for i in range(e):
    u, v, cost = input().split()
    cost = int(cost)
    graph[u].append((v, cost))

print("\nEnter heuristic values:")

for node in nodes:
    heuristic[node] = int(input(f"h({node}) = "))
```



```

start = input("\nEnter Start Node: ")
goal = input("Enter Goal Node: ")
def astar(start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start, [start]))
    visited = set()
    while open_list:
        f, g, current, path = heapq.heappop(open_list)
        if current in visited:
            continue
        visited.add(current)
        if current == goal:
            return path, g
        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                heapq.heappush(open_list, (new_f, new_g, neighbor, path + [neighbor]))
    return None, None
path, cost = astar(start, goal)
print("\nOptimal Path:", " -> ".join(path))
print("Total Path Cost:", cost)

```

Output:

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
= RESTART: C:\Users\New\AppData\Local\Programs\Python\Python313\8 puzzle problem.py
Enter number of nodes: 6

Enter node names:
A
B
C
D
E
G

Enter number of edges: 8

Enter edges (Source Destination Cost):
A B 2
A C 4
B C 3
B D 7
B E 2
C E 3
D E 2
E G 2

Enter heuristic values:
h(A) = 7
h(B) = 6
h(C) = 4
h(D) = 3
h(E) = 2
h(G) = 0

Enter Start Node: A
Enter Goal Node: G

Optimal Path: A -> B -> E -> G
Total Path Cost: 6
```

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\New\AppData\Local\Programs\Python\Python313\8 puzzle problem.py
Enter number of nodes: 5

Enter node names:
S
A
B
C
G

Enter number of edges: 6

Enter edges (Source Destination Cost):
S A 1
S B 4
A C 2
B C 1
C G 3
A G 7

Enter heuristic values:
h(S) = 6
h(A) = 4
h(B) = 3
h(C) = 2
h(G) = 0

Enter Start Node: S
Enter Goal Node: G

Optimal Path: S -> A -> C -> G
Total Path Cost: 6
>>>
```

Result

The A* Search Algorithm successfully finds the shortest path from **A** to **G**.

- **Optimal Path:** A → B → E → G
- **Total Path Cost:** 6

This path is selected because it has the minimum total estimated cost ($f = g + h$) among all possible paths.

Question 2: Dry Run of Minimax with Alpha-Beta Pruning (10 Marks)

Consider the following game tree.



Task:

Perform a **dry run of the Minimax algorithm with Alpha-Beta Pruning** by evaluating the tree **from left to right**.

For each node, show:

- Alpha (α)
- Beta (β)
- Selected value
- Any node that gets pruned

Finally,

1. Determine the best move for MAX.
2. State the final minimax value.
3. List the pruned nodes (if any).

Aim

To determine the best move for the MAX player using the Minimax Algorithm with Alpha-Beta Pruning.

Theory

The Minimax Algorithm is a decision-making algorithm used in two-player games. It assumes that:

- **MAX** tries to maximize the score.
- **MIN** tries to minimize the score.

Alpha-Beta Pruning is an optimization technique that reduces the number of nodes evaluated in the Minimax algorithm by eliminating branches that cannot affect the final decision.

Alpha (α)

- Represents the best (highest) value found so far for the MAX player.
- Initially,
[
 $\alpha = -\infty$
]

Beta (β)

- Represents the best (lowest) value found so far for the MIN player.
- Initially,
[
 $\beta = +\infty$
]

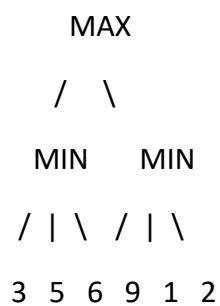
Pruning Condition

If

[
 $\alpha \geq \beta$
]

the remaining child nodes are pruned, because they cannot influence the final decision.

Given Game Tree



Evaluation is performed from left to right.

Initial Values

Node α β

MAX $-\infty$ $+\infty$

Step 1: Evaluate Left MIN Node

Current values:

- $\alpha = -\infty$
- $\beta = +\infty$

The leaf nodes are:

3 , 5 , 6

Evaluate Leaf 3

$$\beta = \min(+\infty, 3)$$

$$\beta = 3$$

Selected value = **3**

Since

[
 $\alpha = -\infty < \beta = 3$
]

Continue.

Evaluate Leaf 5

$$\beta = \min(3, 5)$$

$$\beta = 3$$

Selected value remains **3**.

No pruning occurs.

Evaluate Leaf 6

$$\beta = \min(3, 6)$$

$$\beta = 3$$

Selected value remains **3**.

Left MIN returns **3**.

Update MAX

MAX compares

Current $\alpha = -\infty$

Returned value = 3

Therefore,

$$\alpha = \max(-\infty, 3)$$

$$\alpha = \mathbf{3}$$

Current best move is Left Subtree.

Step 2: Evaluate Right MIN Node

Current values:

- $\alpha = 3$
- $\beta = +\infty$

Leaf nodes are

9 , 1 , 2

Evaluate Leaf 9

$$\beta = \min(+\infty, 9)$$

$$\beta = \mathbf{9}$$

Since

$$3 < 9$$

Continue.

Evaluate Leaf 1

$$\beta = \min(9, 1)$$

$$\beta = \mathbf{1}$$

Now,

[
 $\alpha = 3$
]

[
 $\beta = 1$
]

Since

[
 $\alpha \geq \beta$
]

[
 $3 \geq 1$
]

Alpha-Beta Pruning is applied.

The remaining leaf

2

is not evaluated.

Right MIN returns **1**.

Step 3: Final Decision at MAX

MAX compares

Left subtree = **3**

Right subtree = **1**

MAX chooses

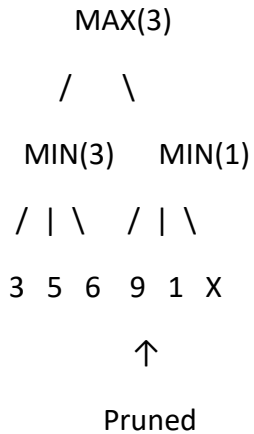
[
 $\max(3, 1) = 3$
]

Dry Run Table

Step	Current Node	α	β	Selected Value	Pruned Node
1	MAX	$-\infty$	$+\infty$	-	None
2	Left MIN	$-\infty$	3	3	None
3	MAX Update	3	$+\infty$	3	None

Step	Current Node	α	β	Selected Value	Pruned Node
4	Right MIN (after 9)	3	9	9	None
5	Right MIN (after 1)	3	1	1	2
6	MAX Final	3	$+\infty$	3	2

Tree Evaluation Summary



(X indicates the pruned node.)

Python Code:

```
import math

# Minimax with Alpha-Beta Pruning

def minimax(depth, nodeIndex, maximizingPlayer, values, alpha, beta):

    # Leaf node

    if depth == 2:

        return values[nodeIndex]

    if maximizingPlayer:

        best = -math.inf

        for i in range(2):

            val = minimax(depth + 1, nodeIndex * 2 + i,

                           False, values, alpha, beta)
```



```

        best = max(best, val)
        alpha = max(alpha, best)
        if beta <= alpha:
            print("Pruning at MAX node")
            break
    return best
else:
    best = math.inf
    for i in range(2):
        val = minimax(depth + 1, nodeIndex * 2 + i,
                       True, values, alpha, beta)
        best = min(best, val)
        beta = min(beta, best)
        if beta <= alpha:
            print("Pruning at MIN node")
            break
    return best

# Main Program
values = []
print("Enter the 4 leaf node values:")
for i in range(4):
    values.append(int(input()))

result = minimax(0, 0, True, values, -math.inf, math.inf)
print("\nFinal Minimax Value =", result)

```

Output:

```
Python 3.11.7 (tags/v3.11.7:7b0ee1c3, Aug 14 2023, 14:19:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\New\AppData\Local\Programs\Python\Python311\8 puzzle problem.py
Enter the 4 leaf node values:
3
5
9
1
Pruning at MIN node
Final Minimax Value = 3

>>> ===== RESTART: C:\Users\New\AppData\Local\Programs\Python\Python311\8 puzzle problem.py =====
Enter the 4 leaf node values:
8
6
4
2
Pruning at MIN node
Final Minimax Value = 6

>>> ===== RESTART: C:\Users\New\AppData\Local\Programs\Python\Python311\8 puzzle problem.py =====
Enter the 4 leaf node values:
7
2
5
4
Pruning at MIN node
Final Minimax Value = 7

>>>
```

Result

The Minimax Algorithm with Alpha-Beta Pruning selects the left subtree because it provides the highest utility value for the MAX player.

- Best Move for MAX: Left MIN subtree
- Final Minimax Value: 3
- Pruned Node: 2 (not evaluated because ($\alpha \geq \beta$))